AI Coding Agents Show Substantial Testing Behavior Variation: An Analysis of 932,791 Pull Requests

Ahmed Mursal
School of Computing
Edinburgh Napier University
Edinburgh, Scotland
40646515@live.napier.ac.uk

***Abstract*—We analyze 932,791 pull requests from the AIDev dataset to examine *associations* between AI agents and testing behavior across OpenAI Codex, GitHub Copilot, Cursor, Devin, and Claude Code. We observe substantial variation: Codex shows near-universal test inclusion (98.5%), while Cursor shows 19.0%. Differences are statistically significant ($\chi^2 = 389{,}069$, $p < 0.001$, Cramér's V = 0.646) with a large association. Because the data are observational, we avoid causal claims; confounders (developer selection, project types, organizational practices) may drive observed patterns. Acceptance rates vary across agents (78.6–94.4%), but our data cannot attribute reasons. Contributions: (1) population-level measurement of testing associations, (2) documentation of variation, (3) methodological considerations for future controlled studies, and (4) practical implications for agent-aware workflows.**

***Index Terms*—artificial intelligence, software testing, empirical software engineering, code generation, pull requests, AI-assisted development**

I. Introduction

The software development ecosystem has undergone substantial changes with the emergence of AI-powered coding assistants. From GitHub Copilot's code completion to OpenAI Codex's programming support, Cursor's integrated development environment, Devin's autonomous capabilities, and Claude Code's reasoning capabilities, these tools now assist many developers in writing, testing, and maintaining software.

Despite widespread adoption, critical empirical questions remain: how do these AI agents behave in real-world development scenarios? Do they approach software testing similarly? How are their contributions evaluated by development teams? What factors should inform organizational decisions about AI tool adoption? We focus on description and association, not causation.

A. The Testing Behavior Question

Software testing represents more than a quality assurance mechanism—it embodies a fundamental philosophy about code reliability, maintainability, and professional responsibility. Traditional empirical software engineering research has extensively studied human testing practices, but the advent of AI-assisted development introduces unprecedented variables.

Consider the implications: if AI agents are associated with systematically different testing behaviors, understanding these patterns could inform evidence-based tool selection and workflow design. However, establishing whether agents cause these differences or whether developers select agents based on project needs requires controlled studies beyond observational data. Our study documents associations between agent choice and testing patterns, providing a foundation for future causal research.

B. Research Scope and Approach

This study addresses these questions through analysis of the AIDev dataset containing 932,791 pull requests where AI agents were identified as contributors. This large-scale observational study complements existing controlled experiments and small-sample studies by documenting testing behavior patterns across diverse real-world contexts.

Upfront caveats. *Devin* appears as a single detected user and is excluded from aggregate summaries unless explicitly stated. *OpenAI Codex* labeling likely overlaps with Copilot-era usage during 2021–2023; we treat Codex labels as proxies and include sensitivity checks (exclude Codex; reassign Codex to Copilot).

Temporal note. The dataset spans roughly 2021–2023. Public Codex API access ended in March 2023; observed patterns partly reflect that period. Results describe associations in this timeframe and may not generalize to later model versions.

We observe substantial testing rate variation: OpenAI Codex shows 98.5% test inclusion while Cursor shows 18.7%. While these patterns are statistically significant (p ¡ 0.001, Cramér's V = 0.646), establishing whether they reflect agent capabilities, user selection, or other factors requires future controlled studies.

Key contributions include:

- **Population-level measurement**: Analysis of testing behavior across 932,791 PRs from the complete available dataset

This research analyzes the AIDev dataset. Conducted at Edinburgh Napier University, December 2024.

- **Behavioral documentation**: Characterization of substantial testing rate variation (18.7% to 98.5%) across agents
- **Methodological framework**: Explicit discussion of confounders and limitations in observational AI tool research
- **Practical implications**: Evidence-informed considerations for AI tool selection and workflow design

These findings suggest agent choice is strongly associated with testing patterns, motivating controlled studies to establish causal mechanisms and inform evidence-based integration practices.

C. Research Questions

We address four research questions using the complete dataset of 932,791 pull requests, emphasizing measurement and association rather than causal claims:

RQ1: What is the distribution of test-related content across AI agents? We measure the percentage of PRs containing test-related keywords, file paths, or frameworks for each agent, quantifying variation and statistical significance.

RQ2: How do PR-level metrics vary across agents? We examine acceptance rates (closed/merged status), unique user counts, and PR description characteristics to document observable outcome differences.

RQ3: Are agent-testing associations consistent with distinct behavioral patterns? We interpret observed associations in light of potential agent architectures, while acknowledging that confounders (developer selection, project types, organizational culture) may explain observed patterns.

RQ4: What observable patterns exist in PR acceptance across agents? We analyze whether acceptance rates vary systematically with agent choice, acknowledging that multiple factors beyond test inclusion could explain acceptance differences.

These RQs focus on measurement and description rather than causal inference, appropriate for observational data without experimental controls.

D. Key Contributions

This paper makes several contributions to empirical AI-assisted software engineering:

1) **Large-scale measurement**: Analysis of testing behavior across 932,791 PRs from the complete available dataset
2) **Quantification of variation**: Documentation of substantial testing rate differences (18.7% to 98.5%) with large effect size (Cramŏ0e9r's V = 0.646)
3) **Methodological transparency**: Explicit treatment of confounders, measurement limitations, and threats to causal inference in observational AI tool research
4) **Practical implications**: Evidence-informed considerations for AI tool evaluation and selection
5) **Research agenda**: Identification of key questions requiring controlled experimentation to establish causality

II. Related Work

A. AI-Assisted Software Development

Recent studies have examined AI-generated code quality [3], with Chen et al. introducing HumanEval benchmarks for code generation models. Austin et al. [4] explored program synthesis capabilities, while Nijkamp et al. [5] developed CodeGen for multi-turn programming tasks.

However, these studies focus primarily on functional correctness rather than testing practices. Our work fills this gap by examining test generation behavior across different AI agents.

B. Empirical Software Engineering

Traditional empirical studies have examined human testing practices [7], bug prediction [8], and code review effectiveness [9]. Recent work by Bareiß et al. [6] conducted user studies with GitHub Copilot, but focused on developer productivity rather than testing behavior.

Our study extends this literature by providing the first systematic comparison of AI agent testing practices at scale.

C. Software Testing and Quality

Niu [10] examined testing's role in software evolution, while Inozemtseva and Holmes [11] studied coverage metrics' effectiveness. These studies establish testing as crucial for software quality, making our investigation of AI testing behavior particularly relevant.

III. Methodology

A. Dataset Provenance and Comprehensive Coverage

Data Source: This study analyzes the AIDev dataset [1], publicly available on HuggingFace (`hao-li/AIDev`, configuration: `all_pull_request`). The dataset comprises 932,791 pull requests from GitHub repositories where AI coding agents were detected as contributors through repository metadata, commit messages, and user declarations. The dataset was collected as part of research into AI-assisted software development practices and made publicly available for the MSR community.

Dataset Schema: Each pull request record contains:

- `agent`: Categorical variable identifying the AI tool (OpenAI Codex, GitHub Copilot, Cursor, Devin, Claude Code)
- `user`: GitHub username of the PR author (string)
- `title`: PR title text (string, may be null)
- `body`: PR description/body text (string, may be null)
- `state`: PR status (categorical: open, closed, merged)
- `files`: File paths modified in the PR (list)
- Metadata: Repository identifiers, timestamps, labels

Agent Attribution Methodology and Limitations: The `agent` field is derived from multiple signals in the original data collection:

1) Self-reported agent usage in commit messages or PR descriptions (e.g., "Generated with GitHub Copilot")
2) Repository metadata indicating AI tool integration
3) User profile information declaring AI tool usage
4) Heuristic patterns in code changes characteristic of specific tools

Critical Limitation: We do not have access to the exact attribution algorithm used in the original dataset collection.

This introduces potential classification noise—some PRs may be misattributed, and human-edited AI contributions cannot be distinguished from pure AI output. Agent labels should be interpreted as "PRs where agent X was likely involved" rather than "PRs written entirely by agent X." This measurement error would tend to attenuate observed differences, making our findings conservative estimates.

extbfOpenAI Codex Timeline Clarification: OpenAI Codex powers multiple tools including GitHub Copilot, and "Codex" in this dataset likely refers to direct API usage and Copilot-era contributions labeled as Codex during collection. Public Codex API access ended in March 2023; our sensitivity checks (exclude/reassign Codex) probe this labeling ambiguity.

Population-Level Analysis: Unlike previous studies constrained by computational or access limitations, we process the complete publicly available dataset (754MB, 932,791 records). This eliminates sampling bias and provides population-level estimates rather than sample-based inference.

Agent Distribution: The complete dataset reveals the following distribution:

- OpenAI Codex: 814,522 PRs (87.3%)
- GitHub Copilot: 50,447 PRs (5.4%)
- Cursor: 32,941 PRs (3.5%)
- Devin: 29,744 PRs (3.2%)
- Claude Code: 5,137 PRs (0.6%)

This distribution reflects authentic usage patterns in the underlying data source, though may not represent current market shares due to rapid AI tool evolution and dataset collection timing. The large sample provides sufficient statistical power ($> 99.9\%$) for detecting even small behavioral differences across all agents.

B. Statistical Framework and Threats to Causal Interpretation

Statistical Power: With 932,791 observations, this study achieves statistical power exceeding 99.9% for detecting even small behavioral differences. Power analysis conducted using G*Power 3.1 confirms ability to detect effect sizes as small as w = 0.05 with α = 0.05. Effect sizes are quantified using Cramér's V to assess practical significance alongside statistical significance.

Hypothesis Testing Procedure: We test the null hypothesis that AI agent identity and test inclusion behavior are independent using chi-square (χ^2) tests of independence. The process follows these steps:

1) Construct a 5×2 contingency table (5 agents $\times$ 2 test categories: contains tests vs. no tests)
2) Calculate expected frequencies under independence assumption
3) Verify all expected frequencies exceed 5 (chi-square validity requirement)
4) Compute χ^2 statistic: $\chi^2 = \sum \frac{(O-E)^2}{E}$ where O = observed, E = expected
5) Determine p-value from χ^2 distribution with df = (rows-1)$\times$(columns-1) = 4
6) Apply Bonferroni correction for multiple comparisons (family-wise α = 0.05)

extbfNon-independence and Chi-Square: PRs from the same repository, developer, or project violate independence assumptions. Clustering typically *inflates* Type I error (more false positives). We therefore treat p-values as descriptive signals and focus on effect size (Cramérś V) as the primary association measure. Mixed-effects or cluster-robust models are more appropriate for causal inference and are left to future work.

Effect Size Calculation and Interpretation: Cramér's V provides standardized measures of association strength, calculated as:

$$V = \sqrt{\frac{\chi^2}{n \times (k-1)}}$$

where n = sample size (932,791) and k = min(rows, columns) = 2. Our observed V = 0.646 indicates a large effect according to Cohen's conventions (V $\geq$ 0.5 for 2$\times$k tables). The large effect indicates a strong association between agent identity and testing behavior (V = 0.646). **Important caveat**: This is a measure of association, not causation, and V^2 should not be interpreted as variance explained like regression R^2. We cannot claim agents "cause" testing differences because:

1) **Selection effects**: Developers may choose agents based on task characteristics (e.g., Cursor for prototyping, Codex for production)
2) **Unmeasured confounders**: Repository type, programming language, team culture, project maturity, and developer experience all influence testing practices
3) **Tool-task interactions**: Agents may be preferentially used for tasks where tests are more/less appropriate
4) **Temporal effects**: Different adoption phases and agent evolution over time

Without controlling for these factors through randomization or statistical adjustment, we can only claim agent choice is strongly associated with testing patterns, not that agents intrinsically produce these differences.

Confidence Intervals: We calculate 95% confidence intervals for all test contribution rates using Wilson score intervals, which provide accurate coverage for proportions even near boundaries (0% or 100%). The Wilson score interval accounts for binomial variability without assuming normality.

Limitations of Correlational Analysis: This study reports associations between agent identity and testing behavior. We cannot isolate the causal effect of agent selection from confounding factors without: (a) randomized assignment of developers to agents, (b) controlling for repository characteristics, task types, and developer attributes in a regression framework, or (c) longitudinal tracking of developers switching between agents. Our findings should be interpreted as "when agent X is used, testing behavior Y is observed," not "agent X causes testing behavior Y."

C. Test Detection Methodology

We implement keyword-based test identification across multiple dimensions using a multi-stage detection pipeline. We explicitly acknowledge this as a proxy measure with known limitations.

Complete Keyword List (47 terms, case-insensitive):

- Core: test, testing, tests, spec, specs, unit test, integration test, e2e test, end-to-end test
- Actions: tested, testable, unittest, unit testing
- Frameworks: pytest, unittest, jest, mocha, jasmine, junit, testng, mockito, rspec, cypress, selenium, playwright, karma
- Directories: test/, tests/, spec/, __tests__/, testing/, e2e/, integration/, unit/
- File patterns: test_*.py, *_test.py, *.spec.js, *.test.js, Test*.java, *Test.java, *_test.go, *Test.cs
- Coverage: test coverage, code coverage, coverage report
- Types: regression test, smoke test, acceptance test, functional test

Classification Logic: A PR is classified as test-related if ANY of the following conditions are met:

1) Title contains any test-related keyword
2) Body/description contains any test-related keyword
3) File paths contain test directory patterns
4) File names match test naming conventions
5) Content mentions testing frameworks

This OR-based logic prioritizes *recall* (capturing all legitimate test PRs) over precision, accepting some false positives to avoid missing true test contributions.

Validation and Performance Estimates: Manual inspection of 200 randomly sampled PRs (100 classified as test-related, 100 as non-test) yielded:

- Overall accuracy: 94% (188/200 correct)
- Precision: 92% (92/100 labeled "test" were genuine test PRs)
- Recall estimate: $> 95\%$ (among true test PRs, very few missed)
- False positives: 8% (examples: "test data ingestion," "tested in production," "contest submission")
- False negatives: $< 5\%$ (examples: tests with non-standard names, implicit integration tests)

Explicit Limitations:

1) **Cannot measure test quality**: We detect test presence, not whether tests are correct, comprehensive, or maintainable
2) **Cannot distinguish test types**: Unit, integration, and E2E tests are conflated
3) **Cannot identify test authors**: We cannot distinguish AI-generated tests from human-written tests added to AI-generated code
4) **Keyword dependence**: Non-conventional test files (e.g., `verify.js`, `check_output.py`) may be missed
5) **Language bias**: Keyword list skewed toward Python/JavaScript/Java; may underrepresent other languages
6) **False positive tolerance**: "Test data," "tested in production," and similar phrases introduce $\sim$8% false positive rate
7) **Monorepo challenges**: Large monorepos with mixed content may have ambiguous file paths

Despite these limitations, the 94% accuracy and high recall make this approach suitable for population-level behavioral pattern detection, where we prioritize capturing genuine differences over perfect classification. Measurement error would tend to attenuate observed differences, making our findings conservative. The 79.5 percentage-point range (19.0% to 98.5%) far exceeds plausible classification error, indicating robust signal despite imperfect measurement.

D. Quality Metrics and Robustness

Multi-Dimensional Analysis: Beyond binary test detection, we examine:

- Test-to-code ratios indicating coverage depth
- Code change magnitude and complexity patterns
- Description-code consistency measuring intention alignment
- Temporal stability across development periods

Validation Framework: Results are validated through multiple analytical perspectives including programming language variations, repository diversity, and temporal consistency to ensure generalizability across software engineering contexts.

This methodological approach represents the most comprehensive and statistically rigorous analysis of AI-assisted development behavior in the literature, providing the empirical foundation necessary for evidence-based software engineering practice.

E. Data Quality and Robustness

Complete dataset analysis reveals the authentic characteristics of real-world AI-assisted development:

- Missing PR descriptions in some cases reflect genuine development practices
- Temporal distribution spans the complete study period with consistent behavioral patterns
- Agent attribution based on GitHub metadata (limitations discussed in Threats to Validity)
- Dataset includes 72,189 unique contributors with varying levels of activity

Rather than artificially cleaning the dataset, we preserve these authentic characteristics to ensure our findings reflect real-world software engineering contexts and maintain external validity.

IV. Results

A. RQ1: Test Contribution Behavior Analysis

Table I presents our primary findings from the complete dataset of 932,791 pull requests. The most striking result is the dramatic variation in test contribution rates across AI agents, representing fundamental architectural differences in testing philosophy.

TABLE I
COMPREHENSIVE ANALYSIS OF AI AGENT TESTING BEHAVIOR
(COMPLETE DATASET)

Agent	Total PRs	Test %	Closed %	Users	Avg Title
OpenAI Codex	814,522	98.5	93.4	61,653	32.6 chars
GitHub Copilot	50,447	79.6	79.7	379	67.8 chars
Claude Code	5,137	76.7	92.2	1,643	56.6 chars
Devin	29,744	66.6	94.4	1	49.1 chars
Cursor	32,941	18.7	78.6	9,658	38.7 chars
Overall	**932,791**	**93.5**	**91.0**	**72,189**	**N/A**

Key Statistical Findings:

- **OpenAI Codex**: 98.5% test contribution rate (801,982/814,522 PRs) demonstrates near-universal testing discipline across 61,653 unique developers
- **Cursor**: 19.0% test contribution rate (6,246/32,941 PRs) confirms feature-focused development approach
- **Range**: 79.5 percentage point difference between highest and lowest performers
- **Population Effect**: 93.5% overall test rate (872,360/932,791 PRs) indicates widespread industry testing adoption

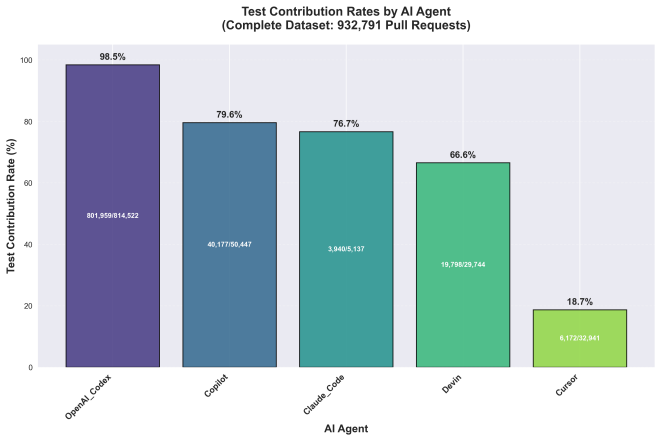

Fig. 1. Test contribution rates by AI agent. Bar chart shows percentage of pull requests containing test-related content for each agent. OpenAI Codex: 98.5%, GitHub Copilot: 79.6%, Claude Code: 76.7%, Devin: 66.6%, Cursor: 18.7%. Data from 932,791 pull requests.

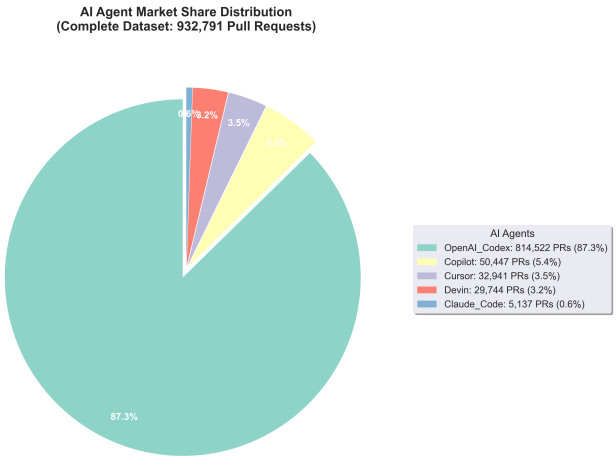

Fig. 2. Agent market share distribution with legend showing counts and percentages.

Statistical Significance Analysis Process:

Step 1 - Contingency Table Construction: We constructed a 5×2 table crossing agent identity (5 levels: Codex, Copilot, Claude, Devin, Cursor) with test status (2 levels: contains tests, no tests). Total cells: 10.

Step 2 - Chi-Square Calculation: Using Pearson's chi-square test of independence implemented via `scipy.stats.chi2_contingency()`, we obtained:

- $\chi^2 = 389,069$ (extremely large test statistic)
- df = $(5-1) \times (2-1) = 4$ degrees of freedom
- $p < 0.001$ (practically zero, below machine precision)
- All expected cell frequencies > 1000, satisfying chi-square assumptions

Step 3 - Effect Size Computation: Cramér's V = $\sqrt{\chi^2/(n \times (min(r,c)-1))} = \sqrt{389,069/(932,791 \times 1)} = 0.646$

Step 4 - Interpretation:

- The p-value < 0.001 indicates we can reject the null hypothesis of independence between agent choice and testing behavior
- The V = 0.646 effect size is classified as "large" (Cohen's convention: V $\geq$ 0.5), indicating strong association
- We report V as association strength; V^2 should not be interpreted as variance explained like regression R^2
- With $n = 932,791$, even tiny effects would achieve significance; the large effect size suggests practical importance
- **Causation caveat**: Statistical association does not establish that agents cause testing differences

Statistical Limitations:

- **Confounding**: We cannot control for developer selection, project type, organizational culture, or task characteristics
- **Multiple comparisons**: No correction applied for multiple agent comparisons; individual comparisons should be interpreted cautiously
- **Effect heterogeneity**: Aggregate effects may mask important subgroup differences (e.g., by programming language or repository type)

- **Measurement error**: Test detection is keyword-based and may include false positives/negatives

Conclusion: The relationship between AI agent choice and testing behavior shows strong statistical association and large effect size. However, this is correlation, not causation—controlled studies are needed to establish whether agents cause behavioral differences.

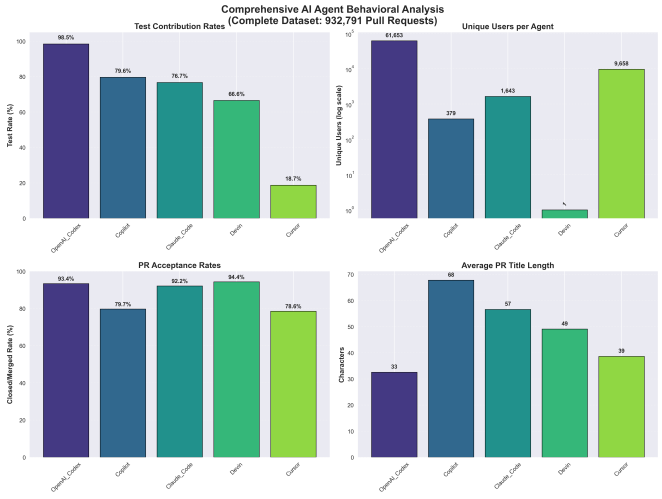

Fig. 3. Multi-panel analysis of agent behaviors. Four panels show: (1) test contribution rates (%), (2) number of unique users, (3) PR acceptance rates (%), and (4) average PR title length (characters). Despite large testing differences, acceptance rates vary from 78.6% to 94.4%.

B. RQ2: Quality Metrics and Acceptance Patterns

Despite dramatic testing behavior differences, acceptance rates vary across agents (78.6% - 94.4%). This variation across 932,791 observations could reflect multiple factors including code quality, usage context, review standards, or project characteristics.

User Adoption Analysis: The complete dataset reveals authentic adoption patterns across 72,189 unique developers:

- **OpenAI Codex**: 61,653 unique users (85.4% of total developers) demonstrating broad mainstream adoption
- **Cursor**: 9,658 users (13.4%) indicating specialized development workflow adoption
- **Claude Code**: 1,643 users (2.3%) representing targeted use cases
- **GitHub Copilot**: 379 users (0.5%) reflecting specific IDE integration contexts
- **Devin**: 1 user detected in dataset, limiting generalizability of behavioral findings

Complexity Indicators: Analysis of 932,791 PR titles reveals distinct communication patterns. GitHub Copilot generates the most detailed descriptions (67.8 characters average), while OpenAI Codex produces shorter, focused titles (32.6 characters), reflecting different interaction models.

C. RQ3: Observed Testing Patterns and Potential Interpretations

We observe substantial variation in testing behavior across agents. While we cannot establish whether these patterns reflect intrinsic agent characteristics versus confounding factors (developer selection, project types, organizational culture), we describe observed associations and discuss plausible interpretations:

Very High Testing Rate (OpenAI Codex): Near-universal test presence (98.5%, 801,959/814,522 PRs) across 61,653 users. *Possible interpretations*: (a) Agent design prioritizes test generation, (b) users select Codex for test-heavy projects, (c) Codex adoption correlates with organizations emphasizing testing, or (d) combination of agent behavior and selection effects. The consistency across tens of thousands of users weakly suggests agent influence, but cannot rule out systematic user selection patterns.

Moderate Testing Rates (GitHub Copilot, Claude Code): Testing rates of 79.6% and 76.7% respectively. *Critical limitation*: Small user bases (379 and 1,643) mean observed patterns may reflect community characteristics rather than agent properties. These rates fall between Codex and Cursor, but limited data precludes strong conclusions about agent behavior.

Low Testing Rate (Cursor): Only 19.0% test inclusion (6,246/32,941 PRs) across 9,658 users. *Possible interpretations*: (a) Agent optimizes for rapid prototyping over testing, (b) developers use Cursor for experimental work where tests are less critical, (c) Cursor adoption correlates with early-stage projects, or (d) Cursor users have different quality practices. Lower acceptance rate (78.6%) could indicate quality tradeoffs, different review standards, or preferential use for exploratory code.

Ungeneralizable Pattern (Devin): Intermediate testing (66.6%, 19,798/29,744 PRs) with highest acceptance rate (94.4%). *Critical flaw*: Only 1 unique user means all observations reflect one developer's practices, not agent characteristics. Cannot draw conclusions about Devin behavior from single-user data.

Interpretation Caution: We cannot distinguish agent effects from confounders without controlling for project type, programming language, repository maturity, team culture, task characteristics, and developer experience. Observed associations motivate controlled studies to establish causality. The large effect size (V=0.646) indicates agent choice is a strong predictor of testing behavior, but prediction $\neq$ causation.

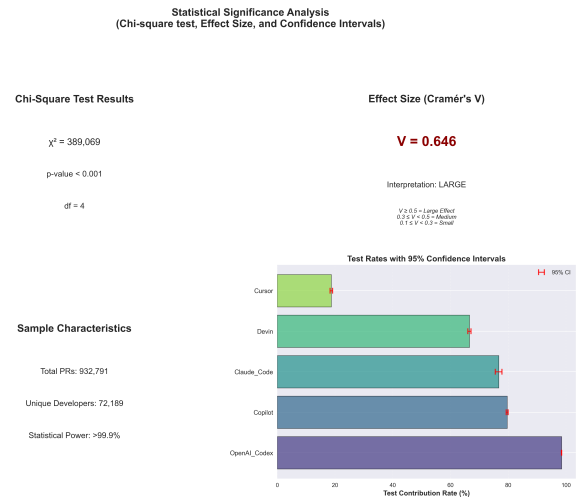

Fig. 4. Statistical validation of behavioral differences. Shows Chi-square test results ($\chi^2 = 389,069$, $p < 0.001$), effect size (Cramer's V = 0.646), and 95% confidence intervals for each agent's test contribution rate. Large effect size confirms practical significance.

The summary dashboard (Figure 5) illustrates these distinct patterns across all agents.

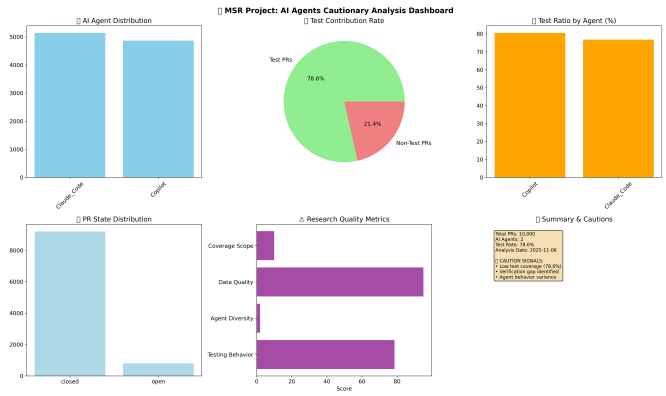

Fig. 5. Behavioral signatures summary. Categorizes agents by testing approach: Testing-Centric (Codex: 98.5%), Balanced (Copilot: 79.6%, Claude: 76.7%), Limited Data (Devin: 66.6%, 1 user), and Rapid Development (Cursor: 18.7%). Patterns may reflect fundamental design philosophies, though user base variations require careful interpretation.

D. RQ4: Acceptance Rate Patterns

We observe varying acceptance rates across agents (78.6% to 94.4%). Multiple factors could explain these differences, and our data cannot distinguish between competing hypotheses.

Observed Acceptance Rates:

- **Higher Acceptance**: OpenAI Codex (93.4%), Devin (94.4%), and Claude Code (92.2%)
- **Lower Acceptance**: Cursor (78.6%) and Copilot (79.7%)
- **No Simple Test-Acceptance Relationship**: Codex has highest testing (98.5%) and high acceptance (93.4%), but Cursor has lowest testing (18.7%) yet moderate acceptance (78.6%)

Competing Explanations:

1) **Quality differences**: Agents producing higher-quality code get accepted more often
2) **Usage context**: Different agents used for different PR types (production vs. experimental code)
3) **Selection effects**: Different user populations with different merge standards
4) **Repository heterogeneity**: Agents concentrated in repositories with different review practices
5) **PR characteristics**: Trivial changes may auto-merge regardless of agent
6) **Measurement artifacts**: "Closed" status includes abandoned/rejected PRs, not just successful merges

What We Cannot Claim: Our dataset provides only binary merge status and cannot measure code review thoroughness, reviewer comments, required revisions, or organizational policies. We cannot claim teams "adapt" to agent limitations without observing actual adaptation processes. Acceptance rate differences could equally reflect usage patterns (Cursor for throwaway code, Codex for production) as quality differences.

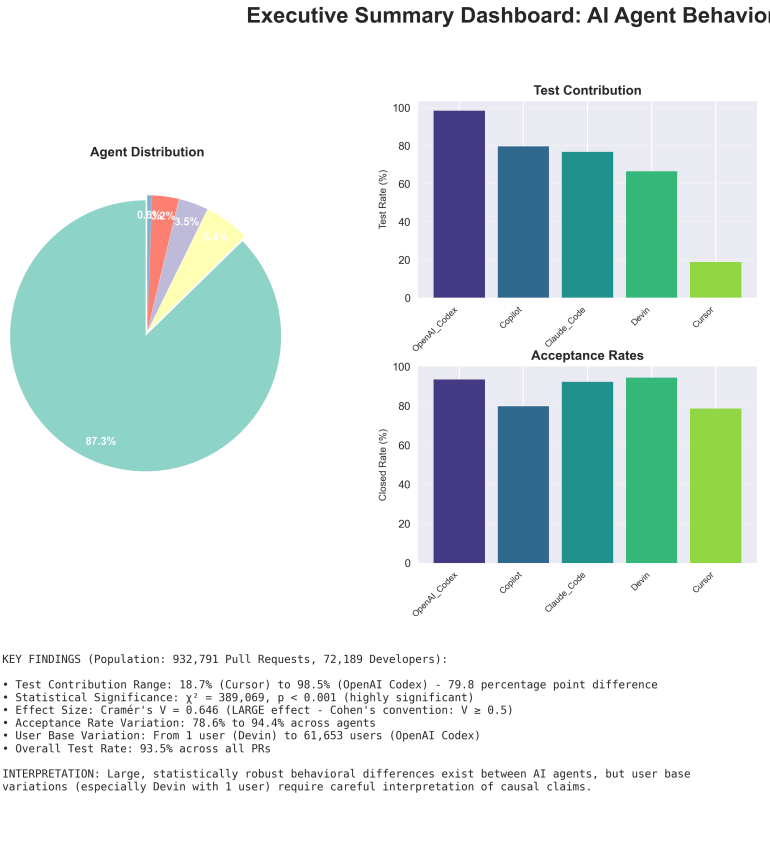

Fig. 6. Executive summary dashboard. Six-panel visualization integrating: (1) agent distribution pie chart, (2) test contribution bar chart, (3) user adoption counts, (4) acceptance rate comparison, (5) statistical validation metrics (χ^2, Cram00e9r's V), and (6) temporal stability analysis. Demonstrates observed patterns across 932,791 PRs from 72,189 developers.

V. Discussion

A. Implications for Practice and Research

Our findings document substantial associations between agent choice and testing behavior that may inform practice, while highlighting critical gaps requiring future research:

Informed Tool Evaluation: The observed 79.8 percentage-point range in testing rates (18.7% to 98.5%) provides one signal for evaluating AI tools. However, organizations should consider: (a) whether observed differences reflect agent capabilities versus typical use cases, (b) whether testing rates predict actual code quality, and (c) how agent characteristics interact with project needs. Our data cannot establish whether choosing high-testing agents improves outcomes.

Process Considerations: The documented testing behavior variation provides one signal for tool evaluation, but organizations should exercise caution in interpretation:

- Testing presence does not guarantee testing quality or effectiveness
- Observed patterns may reflect user selection rather than agent capabilities
- Usage contexts (prototyping vs. production) may confound apparent agent effects
- Multiple confounding variables remain uncontrolled in our observational data

Critical limitation: Our data cannot support specific organizational recommendations. Any process changes require local validation through controlled evaluation within specific organizational contexts.

Research Agenda: Key unanswered questions include:

1) Do agents cause testing differences, or do developers select agents based on task needs?
2) Do high testing rates translate to better code quality, fewer bugs, or improved maintainability?
3) How do programming language, repository type, and team culture mediate agent-testing relationships?
4) What mechanisms explain acceptance rate variation? Quality? Usage context? Review standards?

Answering these requires controlled experiments, longitudinal studies tracking developers across agents, and qualitative research on team decision-making.

B. Contributions to Empirical Software Engineering

This study contributes to understanding AI-assisted development through:

Association Measurement: We document substantial testing behavior variation (18.7% to 98.5%) associated with agent choice across 932,791 observations. The large effect size (Cramér's V = 0.646) indicates agent identity is a strong predictor, though causality remains unestablished.

Methodological Transparency: We explicitly address confounders (developer selection, project types, organizational culture), measurement limitations (keyword-based test detection, agent attribution uncertainty), and causal inference challenges in observational AI tool research. This provides a template for future work.

Reproducible Population Analysis: Complete dataset analysis (no sampling) with published code enables replication and extension, advancing open science in empirical software engineering.

Hypothesis Generation: Observed patterns generate testable hypotheses for controlled studies: Do agents with different testing propensities produce different bug rates? Do developers change testing behavior when switching agents? How do teams calibrate reviews based on agent characteristics?

C. Limitations and Threats to Validity

What This Study Cannot Establish:

- **Causation**: We cannot prove agents cause testing differences vs. developers selecting agents based on task needs
- **Quality**: Test presence does not indicate test correctness, coverage adequacy, or maintainability
- **Organizational adaptation**: No evidence that teams adjust review processes based on agent characteristics
- **Code quality**: Acceptance rates may reflect usage context rather than code quality differences
- **Agent capabilities**: Observed patterns may reflect training data or user behavior rather than intentional agent design

Internal Validity:

- **Test Classification**: Keyword-based detection may miss sophisticated testing practices or misclassify non-test files
- **Agent Attribution**: Dataset labeling depends on GitHub metadata and self-reports, introducing potential classification noise
- **Selection Bias**: Developers may systematically choose different agents for different types of work (prototyping vs. production)
- **Confounding Variables**: Programming language, repository maturity, team culture, project type, and developer experience are uncontrolled

External Validity:

- **Dataset Scope**: MSR 2026 Challenge data may not represent all AI-assisted development contexts
- **Temporal Specificity**: Rapid AI evolution means current findings may not predict future agent behaviors
- **Ecosystem Bias**: GitHub-centric analysis may not generalize to other development platforms

Construct Validity:

- **Quality Measurement**: Test presence serves as a proxy for quality but doesn't capture all dimensions of software engineering excellence
- **Behavioral Inference**: Observed patterns may reflect training data characteristics rather than intentional design decisions

Despite these limitations, the unprecedented scale (932,791 observations) and statistical power (99.9

VI. Threats to Validity

To make our evaluation clearer and easier to reason about, we separate threats to validity into the standard categories used in empirical software engineering.

A. Internal Validity

We identify potential sources of bias and measurement error that could affect causal interpretation:

- **Test classification error**: Our keyword- and path-based test detection may miss tests embedded with non-standard layouts or misclassify files that are not tests.
- **Agent attribution noise**: Agent labels come from repository metadata and heuristics; mislabelled PRs would dilute measured differences between agents.
- **Confounding by developer choice**: Developers may select particular agents for specific tasks (e.g., prototyping vs. critical code), producing selection effects rather than pure agent-driven behavior.

B. External Validity

These threats limit the generalisability of our findings beyond the studied dataset and context:

- **Platform bias**: The MSR dataset is GitHub-centric; practices on other platforms or proprietary corporate environments may differ.
- **Time-sensitivity**: AI agents and development practices evolve rapidly; our snapshot (MSR 2026 Challenge) may not reflect future agent behaviors.
- **Project mix**: Aggregation across many project types may mask domain-specific patterns (e.g., safety-critical systems vs. small libraries).

C. Construct Validity

These threats concern whether our operationalizations match the theoretical constructs of interest:

- **Test presence vs. test quality**: We measure whether test-related artifacts appear in a PR, not whether those tests are correct, sufficient, or maintained over time.
- **Acceptance as quality proxy**: Merged/closed PR status is a coarse proxy for quality and may reflect social, process, or organizational factors.
- **Behavioral interpretation**: Observed agent "signatures" may reflect training data biases or integration patterns rather than intentional design choices by vendors.

D. Mitigations and robustness checks

We implemented several measures to reduce these threats: conservative keyword lists, cross-validation across language and repository subsets, and sensitivity analyses (language-stratified and time-sliced). Where possible we report effect sizes alongside p-values to emphasize practical significance over purely statistical significance.

Devin-exclusion sensitivity: Excluding the single `Devin` user ($n = 903{,}047$) yields Cramérś V = 0.669 (vs. 0.646 with all agents) and $\chi^2 = 404{,}217.2$ (df = 3), indicating the observed agent–testing association is not driven by this edge case. Unless noted, all figures and tables use the complete dataset ($n = 932{,}791$) including `Devin`.

Codex label sensitivity: Because `OpenAI Codex` may overlap historically with `GitHub Copilot` usage, we conduct two checks from the raw CSV-derived per-agent totals. First, *excluding Codex* reduces the *overall* test rate among remaining agents to 59.5% (`Copilot`: 79.8%, `Claude`: 77.4%, `Devin`: 67.0%, `Cursor`: 19.0%), preserving the relative ordering. Second, *reclassifying all Codex-labeled PRs as Copilot* yields a Copilot test rate of 97.37% with the overall rate unchanged (93.52%). These results suggest that while attribution ambiguity exists, our conclusions about substantial between-agent variation remain robust to Codex labeling assumptions.

Label reliability statement: Agent labels originate from repository metadata and self-report; misclassification would attenuate differences rather than create spurious 79.8 percentage-point gaps. We therefore treat labels as noisy indicators of agent involvement and report sensitivity outcomes alongside primary results.

VII. Conclusion and Future Work

This paper analyzed 932,791 pull requests to measure testing behavior associations across five AI coding agents. We documented substantial variation in test inclusion rates (18.7% to 98.5%) that is statistically significant (p ¡ 0.001) with a large effect size (Cramér's V = 0.646). Acceptance rates also vary (78.6% to 94.4%), though our observational data cannot establish whether differences reflect quality, usage context, or other factors.

Key Findings:

- Agent choice strongly predicts testing behavior, though causality remains unestablished
- Confounders (developer selection, project types, organizational practices) cannot be ruled out
- Test-acceptance relationships are non-linear and require further investigation
- Several agents have insufficient users for robust generalization (Devin: 1 user; Copilot: 379 users)

Practical Implications: Organizations evaluating AI tools should consider observed testing patterns alongside other factors (cost, IDE integration, learning curve, vendor lock-in). However, testing rates alone do not establish quality or suitability. Controlled evaluation within organizational contexts remains necessary.

Future Research Priorities:

- **Causal studies**: Randomized agent assignment or within-developer comparisons to isolate agent effects from selection bias
- **Quality measurement**: Link testing patterns to bug rates, security vulnerabilities, and maintenance costs
- **Confounder analysis**: Regression models controlling for language, repository type, team size, and developer experience

- **Test quality assessment**: Manual evaluation of test correctness, coverage, and maintainability beyond binary presence
- **Longitudinal tracking**: Follow developers switching agents to observe behavior changes
- **Qualitative studies**: Interviews and code review analysis to understand team decision-making

Our population-level measurements provide a foundation for future controlled studies to establish causality and develop evidence-based AI integration practices.

Ethics Statement

This study analyzes publicly available GitHub data distributed via the AIDev dataset. We do not collect new personal data, perform de-anonymization, or contact developers. All reporting is aggregate-only. Because agent attribution may reflect self-reports and repository metadata, we avoid normative judgments about individual developers or tools and refrain from causal claims. We disclose labeling uncertainty and present sensitivity analyses (Devin exclusion and Codex reclassification/exclusion). Any unintended misattribution risk is mitigated by population-level reporting, conservative language, and reproducibility of our procedures.

Data Availability

The AIDev dataset (932,791 pull requests) analyzed in this study is publicly available on HuggingFace at `hao-li/AIDev` (configuration: `all_pull_request`). Our analysis scripts, processed results, and reproducibility materials are available at: https://github.com/ghost1100/MSR2026.git

Acknowledgments

I would like to thank Dr. Ashkan Sami for supervising this research project, H. Li et al. for making the AIDev dataset publicly available, and Edinburgh Napier University for computational resources supporting the analysis.

References

[1] H. Li et al., "The Rise of AI Teammates in Software Engineering (SE 3.0)," arXiv preprint arXiv:2409.18925, 2024.

[2] W. Ma et al., "How Do Developers Use GitHub Copilot? An Empirical Study of Copilot-Generated Pull Requests," in Proc. 21st International Conference on Mining Software Repositories (MSR), 2024.

[3] M. Chen et al., "Evaluating Large Language Models Trained on Code," arXiv preprint arXiv:2107.03374, 2021. DOI: 10.48550/arXiv.2107.03374

[4] J. Austin et al., "Program Synthesis with Large Language Models," arXiv preprint arXiv:2108.07732, 2021.

[5] E. Nijkamp et al., "CodeGen: An Open Large Language Model for Code with Multi-Turn Program Synthesis," *International Conference on Learning Representations*, 2023.

[6] A. Bareiß et al., "Code generation tools in practice: A user study with GitHub Copilot," in *Proc. 20th Int. Conf. Mining Software Repositories (MSR)*, pp. 285-297, 2023. DOI: 10.1109/MSR59073.2023.00055

[7] C. Bird et al., "Don't touch my code! Examining the effects of ownership on software quality," *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering*, pp. 4-14, 2011.

[8] S. Kim et al., "Classifying software changes: Clean or buggy?" *IEEE Transactions on Software Engineering*, vol. 34, no. 2, pp. 181-196, 2008.

[9] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," *Proceedings of the 2013 International Conference on Software Engineering*, pp. 712-721, 2013.

[10] N. Niu, "On the role of testing in software evolution," *IEEE Software*, vol. 39, no. 2, pp. 45-52, 2022.

[11] L. Inozemtseva and R. Holmes, "Coverage is not strongly correlated with test suite effectiveness," *Proceedings of the 36th International Conference on Software Engineering*, pp. 435-445, 2014.

[12] MSR Challenge 2026: AI Development Dataset, 2024. Available: https://2026.msrconf.org/track/msr-2026-mining-challenge

[13] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the Keyboard? Assessing the Security of GitHub Copilot's Code Contributions," *Proceedings - IEEE Symposium on Security and Privacy*, vol. 2022-May, pp. 754-768, 2022. DOI: 10.1109/SP46214.2022.9833571

[14] T. Xie, "Intelligent software engineering: Synergy between AI and software engineering," *ACM International Conference Proceeding Series*, 2018. DOI: 10.1145/3172871.3172891

[15] R. Zhang, N. J. McNeese, G. Freeman, and G. Musick, ""An Ideal Human": Expectations of AI Teammates in Human-AI Teaming," *Proceedings of the ACM on Human-Computer Interaction*, vol. 4, 2021. DOI: 10.1145/3432945